

Bayescoverage APP - overview

Leontine Alkema, last updated June 8, 2026

Installation info is given in the readme and shinyapp

```
library(bayescoveragedeploy)
```

Where data are stored:

```
data_folder <- "data_raw" # for survey data
routinedata_dir <- here::here("private/routinedata/")
# see note on working with routine dummy data below
```

Choose an indicator. Naming comes from survey data, options include

- c("anc1trimester", "vmsl", "ideliv", "vdpt", "anc4") , with routine as available
- c("bfexcl0_5", "ancq8", "cci", "sba"), no routine data

```
indicator_select <- "anc1trimester"#"anc4" #"cci"#"vdpt"#"anc4"#"anc4"#"
```

Read in survey data and process

```
dat0 <- haven::read_dta(here::here(data_folder, "ICEH_all.long_20260603.dta")) |>
  dplyr::rename(r = r_raw, se = se_raw)
```

Fitting for one country, use ISO to select

```
# countries with survey data
# unique(dat$iso)
iso_select <- "KEN" #RWA"
dat_use <-
  bayescoveragemodel::process_data(
    dat = dat0 |>
```

```
dplyr::filter(iso %in% iso_select),
regions_dat = bayescoveragemodel::regions_all,
indicator = indicator_select, verbose = FALSE)
```

```
[1] "Column possible_outlier not included in data, no pre-defined outliers used"
```

Do we want to add routine data?

```
add_routine <- ifelse(
  indicator_select %in% c("anc1trimester", "vmsl", "ideliv", "vdpt", "anc4"),
  TRUE,
  FALSE)
add_routine
```

```
[1] TRUE
```

If yes, read in routine data.

```
if (!add_routine){
  routine_dat <- routine_dat_use <- NULL
} else {
  # # if we want to use real data
  # routine_dat_raw <- readr::read_csv(
  #   here::here("private/routinedata/routine_data_all.csv"),
  #   show_col_types = FALSE)
  # routine_dat_use <- routine_dat_raw |>
  #   dplyr::filter(iso == iso_select, indicator_name == indicator_select)

  # or if we want to use dummy data
  routine_dat_use <- readr::read_csv(here::here("data_raw/routine_toydata.csv")) |>
  # add iso
  dplyr::mutate(iso = iso_select,
                indicator_name = indicator_select)
}
```

```
Rows: 5 Columns: 5
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
dbl (3): year, routine_value, countdownmean
```

```
lgl (2): iso, indicator_name
```

- i Use ``spec()`` to retrieve the full column specification for this data.
- i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
routine_dat_use
```

```
# A tibble: 5 x 5
  iso   indicator_name year routine_value countdownmean
<chr> <chr>          <dbl>         <dbl>         <dbl>
1 KEN   anc1trimester  2020           0.52           95
2 KEN   anc1trimester  2021           0.53          100
3 KEN   anc1trimester  2022           0.55          100
4 KEN   anc1trimester  2023           0.52          100
5 KEN   anc1trimester  2024           0.56          100
```

```
# or use a dummy set as follows:
# routine_dat_use <- data.frame(
#   iso = rep(iso_select, 3),
#   year = 2021:2023,
#   indicator_name = indicator_select,
#   routine_value = c(0.85, 0.87, 0.90),
#   countdownmean = rep(100, 3))
```

```
fit <- bayescoveragedeploy::fit_local_model(
  survey_df = dat_use,
  indicator = indicator_select,
  iso_select = iso_select,
  routine_df = routine_dat_use,
  # these sampling settings are the new defaults in updated deploy package
  # so not needed if you use deploy 0.2 or higher
  iter_sampling = 300,
  iter_warmup = 150,
  adapt_delta = 0.95,
  max_treedepth = 14
)
```

```
[1] "We use a global fit, and take selected settings from there."
[1] "settings for the spline_degree and num_knots taken from global run"
[1] "settings for Ptilde_low taken from global run"
[1] "Setting for tstar taken from global run"
[1] "Settings for hierarchical settings taken from global run"
```

```

[1] "For hierarchical terms, we fix things up to the 2nd-lowest level."
[1] "We fix all sigmas of hierarchical models."
[1] "We take the data model setting from global fit."
[1] "We add smoothing"
[1] "We fix smoothing and data model parameters."
[1] "We filter data to be inside estimation period"
[1] "We use national data"
[1] "We only use data for KEN"
[1] "We fit to survey data from 2010 onwards."
[1] "NOT using or producing aggregates"

```

Joining with `by = join\_by(iso)`

```

[1] "Using precompiled Stan model"

```

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 0.000101 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 1.01 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 450 [0%] (Warmup)

Chain 1: Iteration: 10 / 450 [2%] (Warmup)

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 0.000363 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 3.63 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 450 [0%] (Warmup)

Chain 1: Iteration: 20 / 450 [4%] (Warmup)

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 3).

Chain 3:

Chain 3: Gradient evaluation took 6.9e-05 seconds

Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.69 seconds.

Chain 3: Adjust your expectations accordingly!

Chain 3:

Chain 3:

Chain 3: Iteration: 1 / 450 [0%] (Warmup)
Chain 2: Iteration: 10 / 450 [2%] (Warmup)
Chain 1: Iteration: 30 / 450 [6%] (Warmup)
Chain 3: Iteration: 10 / 450 [2%] (Warmup)

SAMPLING FOR MODEL 'fpem_routine' NOW (CHAIN 4).

Chain 4:

Chain 4: Gradient evaluation took 6.7e-05 seconds

Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.67 seconds.

Chain 4: Adjust your expectations accordingly!

Chain 4:

Chain 4:

Chain 4: Iteration: 1 / 450 [0%] (Warmup)
Chain 1: Iteration: 40 / 450 [8%] (Warmup)
Chain 2: Iteration: 20 / 450 [4%] (Warmup)
Chain 3: Iteration: 20 / 450 [4%] (Warmup)
Chain 4: Iteration: 10 / 450 [2%] (Warmup)
Chain 2: Iteration: 30 / 450 [6%] (Warmup)
Chain 1: Iteration: 50 / 450 [11%] (Warmup)
Chain 3: Iteration: 30 / 450 [6%] (Warmup)
Chain 2: Iteration: 40 / 450 [8%] (Warmup)
Chain 4: Iteration: 20 / 450 [4%] (Warmup)
Chain 3: Iteration: 40 / 450 [8%] (Warmup)
Chain 1: Iteration: 60 / 450 [13%] (Warmup)
Chain 2: Iteration: 50 / 450 [11%] (Warmup)
Chain 4: Iteration: 30 / 450 [6%] (Warmup)
Chain 2: Iteration: 60 / 450 [13%] (Warmup)
Chain 1: Iteration: 70 / 450 [15%] (Warmup)
Chain 3: Iteration: 50 / 450 [11%] (Warmup)
Chain 2: Iteration: 70 / 450 [15%] (Warmup)
Chain 1: Iteration: 80 / 450 [17%] (Warmup)
Chain 4: Iteration: 40 / 450 [8%] (Warmup)
Chain 2: Iteration: 80 / 450 [17%] (Warmup)
Chain 3: Iteration: 60 / 450 [13%] (Warmup)
Chain 1: Iteration: 90 / 450 [20%] (Warmup)
Chain 2: Iteration: 90 / 450 [20%] (Warmup)
Chain 4: Iteration: 50 / 450 [11%] (Warmup)
Chain 3: Iteration: 70 / 450 [15%] (Warmup)
Chain 1: Iteration: 100 / 450 [22%] (Warmup)
Chain 2: Iteration: 100 / 450 [22%] (Warmup)
Chain 3: Iteration: 80 / 450 [17%] (Warmup)
Chain 4: Iteration: 60 / 450 [13%] (Warmup)
Chain 3: Iteration: 90 / 450 [20%] (Warmup)

Chain 4: Iteration: 70 / 450 [15%] (Warmup)
Chain 2: Iteration: 110 / 450 [24%] (Warmup)
Chain 1: Iteration: 110 / 450 [24%] (Warmup)
Chain 3: Iteration: 100 / 450 [22%] (Warmup)
Chain 2: Iteration: 120 / 450 [26%] (Warmup)
Chain 3: Iteration: 110 / 450 [24%] (Warmup)
Chain 4: Iteration: 80 / 450 [17%] (Warmup)
Chain 1: Iteration: 120 / 450 [26%] (Warmup)
Chain 2: Iteration: 130 / 450 [28%] (Warmup)
Chain 3: Iteration: 120 / 450 [26%] (Warmup)
Chain 2: Iteration: 140 / 450 [31%] (Warmup)
Chain 4: Iteration: 90 / 450 [20%] (Warmup)
Chain 1: Iteration: 130 / 450 [28%] (Warmup)
Chain 3: Iteration: 130 / 450 [28%] (Warmup)
Chain 1: Iteration: 140 / 450 [31%] (Warmup)
Chain 4: Iteration: 100 / 450 [22%] (Warmup)
Chain 3: Iteration: 140 / 450 [31%] (Warmup)
Chain 2: Iteration: 150 / 450 [33%] (Warmup)
Chain 2: Iteration: 151 / 450 [33%] (Sampling)
Chain 1: Iteration: 150 / 450 [33%] (Warmup)
Chain 1: Iteration: 151 / 450 [33%] (Sampling)
Chain 2: Iteration: 160 / 450 [35%] (Sampling)
Chain 1: Iteration: 160 / 450 [35%] (Sampling)
Chain 3: Iteration: 150 / 450 [33%] (Warmup)
Chain 3: Iteration: 151 / 450 [33%] (Sampling)
Chain 4: Iteration: 110 / 450 [24%] (Warmup)
Chain 2: Iteration: 170 / 450 [37%] (Sampling)
Chain 1: Iteration: 170 / 450 [37%] (Sampling)
Chain 3: Iteration: 160 / 450 [35%] (Sampling)
Chain 2: Iteration: 180 / 450 [40%] (Sampling)
Chain 1: Iteration: 180 / 450 [40%] (Sampling)
Chain 3: Iteration: 170 / 450 [37%] (Sampling)
Chain 4: Iteration: 120 / 450 [26%] (Warmup)
Chain 1: Iteration: 190 / 450 [42%] (Sampling)
Chain 2: Iteration: 190 / 450 [42%] (Sampling)
Chain 3: Iteration: 180 / 450 [40%] (Sampling)
Chain 4: Iteration: 130 / 450 [28%] (Warmup)
Chain 1: Iteration: 200 / 450 [44%] (Sampling)
Chain 2: Iteration: 200 / 450 [44%] (Sampling)
Chain 3: Iteration: 190 / 450 [42%] (Sampling)
Chain 1: Iteration: 210 / 450 [46%] (Sampling)
Chain 2: Iteration: 210 / 450 [46%] (Sampling)
Chain 3: Iteration: 200 / 450 [44%] (Sampling)

Chain 1: Iteration: 220 / 450 [48%] (Sampling)
Chain 2: Iteration: 220 / 450 [48%] (Sampling)
Chain 4: Iteration: 140 / 450 [31%] (Warmup)
Chain 3: Iteration: 210 / 450 [46%] (Sampling)
Chain 1: Iteration: 230 / 450 [51%] (Sampling)
Chain 2: Iteration: 230 / 450 [51%] (Sampling)
Chain 4: Iteration: 150 / 450 [33%] (Warmup)
Chain 4: Iteration: 151 / 450 [33%] (Sampling)
Chain 3: Iteration: 220 / 450 [48%] (Sampling)
Chain 1: Iteration: 240 / 450 [53%] (Sampling)
Chain 4: Iteration: 160 / 450 [35%] (Sampling)
Chain 2: Iteration: 240 / 450 [53%] (Sampling)
Chain 3: Iteration: 230 / 450 [51%] (Sampling)
Chain 1: Iteration: 250 / 450 [55%] (Sampling)
Chain 4: Iteration: 170 / 450 [37%] (Sampling)
Chain 2: Iteration: 250 / 450 [55%] (Sampling)
Chain 3: Iteration: 240 / 450 [53%] (Sampling)
Chain 1: Iteration: 260 / 450 [57%] (Sampling)
Chain 4: Iteration: 180 / 450 [40%] (Sampling)
Chain 2: Iteration: 260 / 450 [57%] (Sampling)
Chain 3: Iteration: 250 / 450 [55%] (Sampling)
Chain 1: Iteration: 270 / 450 [60%] (Sampling)
Chain 2: Iteration: 270 / 450 [60%] (Sampling)
Chain 3: Iteration: 260 / 450 [57%] (Sampling)
Chain 4: Iteration: 190 / 450 [42%] (Sampling)
Chain 1: Iteration: 280 / 450 [62%] (Sampling)
Chain 2: Iteration: 280 / 450 [62%] (Sampling)
Chain 3: Iteration: 270 / 450 [60%] (Sampling)
Chain 4: Iteration: 200 / 450 [44%] (Sampling)
Chain 1: Iteration: 290 / 450 [64%] (Sampling)
Chain 2: Iteration: 290 / 450 [64%] (Sampling)
Chain 3: Iteration: 280 / 450 [62%] (Sampling)
Chain 4: Iteration: 210 / 450 [46%] (Sampling)
Chain 1: Iteration: 300 / 450 [66%] (Sampling)
Chain 2: Iteration: 300 / 450 [66%] (Sampling)
Chain 3: Iteration: 290 / 450 [64%] (Sampling)
Chain 4: Iteration: 220 / 450 [48%] (Sampling)
Chain 1: Iteration: 310 / 450 [68%] (Sampling)
Chain 2: Iteration: 310 / 450 [68%] (Sampling)
Chain 3: Iteration: 300 / 450 [66%] (Sampling)
Chain 4: Iteration: 230 / 450 [51%] (Sampling)
Chain 1: Iteration: 320 / 450 [71%] (Sampling)
Chain 2: Iteration: 320 / 450 [71%] (Sampling)

Chain 3: Iteration: 310 / 450 [68%] (Sampling)
Chain 4: Iteration: 240 / 450 [53%] (Sampling)
Chain 1: Iteration: 330 / 450 [73%] (Sampling)
Chain 2: Iteration: 330 / 450 [73%] (Sampling)
Chain 3: Iteration: 320 / 450 [71%] (Sampling)
Chain 4: Iteration: 250 / 450 [55%] (Sampling)
Chain 1: Iteration: 340 / 450 [75%] (Sampling)
Chain 2: Iteration: 340 / 450 [75%] (Sampling)
Chain 3: Iteration: 330 / 450 [73%] (Sampling)
Chain 1: Iteration: 350 / 450 [77%] (Sampling)
Chain 4: Iteration: 260 / 450 [57%] (Sampling)
Chain 2: Iteration: 350 / 450 [77%] (Sampling)
Chain 3: Iteration: 340 / 450 [75%] (Sampling)
Chain 1: Iteration: 360 / 450 [80%] (Sampling)
Chain 4: Iteration: 270 / 450 [60%] (Sampling)
Chain 2: Iteration: 360 / 450 [80%] (Sampling)
Chain 3: Iteration: 350 / 450 [77%] (Sampling)
Chain 1: Iteration: 370 / 450 [82%] (Sampling)
Chain 4: Iteration: 280 / 450 [62%] (Sampling)
Chain 3: Iteration: 360 / 450 [80%] (Sampling)
Chain 2: Iteration: 370 / 450 [82%] (Sampling)
Chain 1: Iteration: 380 / 450 [84%] (Sampling)
Chain 4: Iteration: 290 / 450 [64%] (Sampling)
Chain 3: Iteration: 370 / 450 [82%] (Sampling)
Chain 2: Iteration: 380 / 450 [84%] (Sampling)
Chain 1: Iteration: 390 / 450 [86%] (Sampling)
Chain 4: Iteration: 300 / 450 [66%] (Sampling)
Chain 3: Iteration: 380 / 450 [84%] (Sampling)
Chain 2: Iteration: 390 / 450 [86%] (Sampling)
Chain 1: Iteration: 400 / 450 [88%] (Sampling)
Chain 4: Iteration: 310 / 450 [68%] (Sampling)
Chain 3: Iteration: 390 / 450 [86%] (Sampling)
Chain 2: Iteration: 400 / 450 [88%] (Sampling)
Chain 1: Iteration: 410 / 450 [91%] (Sampling)
Chain 4: Iteration: 320 / 450 [71%] (Sampling)
Chain 3: Iteration: 400 / 450 [88%] (Sampling)
Chain 2: Iteration: 410 / 450 [91%] (Sampling)
Chain 1: Iteration: 420 / 450 [93%] (Sampling)
Chain 4: Iteration: 330 / 450 [73%] (Sampling)
Chain 3: Iteration: 410 / 450 [91%] (Sampling)
Chain 2: Iteration: 420 / 450 [93%] (Sampling)
Chain 1: Iteration: 430 / 450 [95%] (Sampling)
Chain 4: Iteration: 340 / 450 [75%] (Sampling)


```

Chain 3: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 2: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 1: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 4: Iteration: 350 / 450 [ 77%] (Sampling)
Chain 3: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 2: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 1: Iteration: 450 / 450 [100%] (Sampling)
Chain 1:
Chain 1: Elapsed Time: 0.397 seconds (Warm-up)
Chain 1:           0.709 seconds (Sampling)
Chain 1:           1.106 seconds (Total)
Chain 1:
Chain 4: Iteration: 360 / 450 [ 80%] (Sampling)
Chain 3: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 2: Iteration: 450 / 450 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 0.37 seconds (Warm-up)
Chain 2:           0.736 seconds (Sampling)
Chain 2:           1.106 seconds (Total)
Chain 2:
Chain 4: Iteration: 370 / 450 [ 82%] (Sampling)
Chain 3: Iteration: 450 / 450 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 0.38 seconds (Warm-up)
Chain 3:           0.73 seconds (Sampling)
Chain 3:           1.11 seconds (Total)
Chain 3:
Chain 4: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 4: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 4: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 4: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 4: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 4: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 4: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 4: Iteration: 450 / 450 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 0.51 seconds (Warm-up)
Chain 4:           0.766 seconds (Sampling)
Chain 4:           1.276 seconds (Total)
Chain 4:

```

Warning: There were 5 divergent transitions after warmup. See

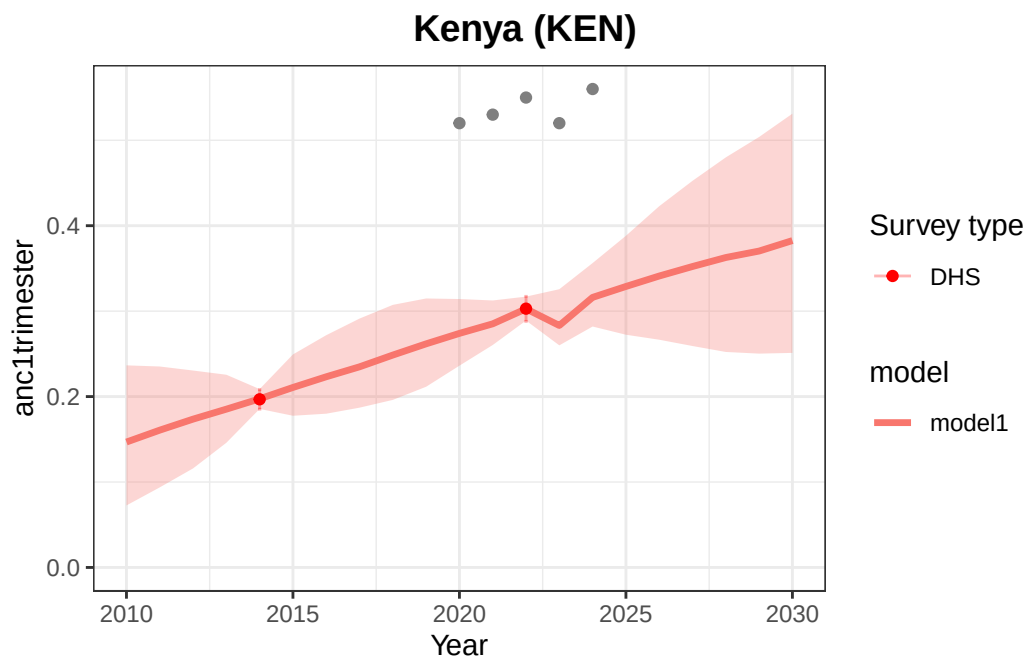
<https://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup>
to find out why this is a problem and how to eliminate them.

Warning: Examine the pairs() plot to diagnose sampling problems

Extracting posteriors...

Overview plot

```
countryplot <- bayescoveragemodel::plot_estimates_local_all(  
  results = fit,  
  save_plots = FALSE)[[1]]  
countryplot
```



Estimates

```
fit$posteriors$temporal |>  
  dplyr::select(iso, year, `50%`, `2.5%`, `97.5%`)
```

```
# A tibble: 21 x 5  
  iso    year `50%` `2.5%` `97.5%`  
  <chr> <int> <dbl> <dbl> <dbl>
```

```

1 KEN    2010 0.147 0.0726 0.237
2 KEN    2011 0.161 0.0937 0.235
3 KEN    2012 0.174 0.116 0.230
4 KEN    2013 0.185 0.146 0.226
5 KEN    2014 0.197 0.186 0.209
6 KEN    2015 0.211 0.178 0.249
7 KEN    2016 0.223 0.180 0.272
8 KEN    2017 0.235 0.187 0.291
9 KEN    2018 0.249 0.196 0.307
10 KEN   2019 0.262 0.212 0.315
# i 11 more rows

```

Compare to a fit w/o routine data (if routine data were added)

```

fit_noroutine<- bayescoveragedeploy::fit_local_model(
  survey_df = dat_use,
  indicator = indicator_select,
  iso_select = iso_select,
  routine_df = NULL,
  # these sampling settings are the new defaults in updated deploy package
  # so not needed if you use deploy 0.2 or higher
  iter_sampling = 300,
  iter_warmup = 150,
  adapt_delta = 0.95,
  max_treedepth = 14
)

```

```

[1] "We use a global fit, and take selected settings from there."
[1] "settings for the spline_degree and num_knots taken from global run"
[1] "settings for Ptilde_low taken from global run"
[1] "Setting for tstar taken from global run"
[1] "Settings for hierarchical settings taken from global run"
[1] "For hierarchical terms, we fix things up to the 2nd-lowest level."
[1] "We fix all sigmas of hierarchical models."
[1] "We take the data model setting from global fit."
[1] "We add smoothing"
[1] "We fix smoothing and data model parameters."
[1] "We filter data to be inside estimation period"
[1] "We use national data"
[1] "We only use data for KEN"
[1] "We fit to survey data from 2010 onwards."
[1] "NOT using or producing aggregates"

```

[1] "Using precompiled Stan model"

SAMPLING FOR MODEL 'fpem' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 8.1e-05 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.81 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 450 [0%] (Warmup)

SAMPLING FOR MODEL 'fpem' NOW (CHAIN 2).

Chain 1: Iteration: 10 / 450 [2%] (Warmup)

Chain 2:

Chain 2: Gradient evaluation took 9.1e-05 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.91 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 450 [0%] (Warmup)

SAMPLING FOR MODEL 'fpem' NOW (CHAIN 3).

SAMPLING FOR MODEL 'fpem' NOW (CHAIN 4).

Chain 3:

Chain 3: Gradient evaluation took 7.2e-05 seconds

Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.72 seconds.

Chain 3: Adjust your expectations accordingly!

Chain 3:

Chain 3:

Chain 3: Iteration: 1 / 450 [0%] (Warmup)

Chain 4:

Chain 4: Gradient evaluation took 0.000272 seconds

Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 2.72 seconds.

Chain 4: Adjust your expectations accordingly!

Chain 4:

Chain 4:

Chain 4: Iteration: 1 / 450 [0%] (Warmup)

Chain 1: Iteration: 20 / 450 [4%] (Warmup)

Chain 4: Iteration: 10 / 450 [2%] (Warmup)

Chain 1: Iteration: 30 / 450 [6%] (Warmup)

Chain 4: Iteration: 20 / 450 [4%] (Warmup)

Chain 3: Iteration: 10 / 450 [2%] (Warmup)

Chain 1: Iteration: 40 / 450 [8%] (Warmup)
Chain 2: Iteration: 10 / 450 [2%] (Warmup)
Chain 4: Iteration: 30 / 450 [6%] (Warmup)
Chain 3: Iteration: 20 / 450 [4%] (Warmup)
Chain 1: Iteration: 50 / 450 [11%] (Warmup)
Chain 4: Iteration: 40 / 450 [8%] (Warmup)
Chain 1: Iteration: 60 / 450 [13%] (Warmup)
Chain 4: Iteration: 50 / 450 [11%] (Warmup)
Chain 3: Iteration: 30 / 450 [6%] (Warmup)
Chain 1: Iteration: 70 / 450 [15%] (Warmup)
Chain 2: Iteration: 20 / 450 [4%] (Warmup)
Chain 4: Iteration: 60 / 450 [13%] (Warmup)
Chain 3: Iteration: 40 / 450 [8%] (Warmup)
Chain 1: Iteration: 80 / 450 [17%] (Warmup)
Chain 2: Iteration: 30 / 450 [6%] (Warmup)
Chain 4: Iteration: 70 / 450 [15%] (Warmup)
Chain 3: Iteration: 50 / 450 [11%] (Warmup)
Chain 2: Iteration: 40 / 450 [8%] (Warmup)
Chain 4: Iteration: 80 / 450 [17%] (Warmup)
Chain 1: Iteration: 90 / 450 [20%] (Warmup)
Chain 1: Iteration: 100 / 450 [22%] (Warmup)
Chain 4: Iteration: 90 / 450 [20%] (Warmup)
Chain 3: Iteration: 60 / 450 [13%] (Warmup)
Chain 2: Iteration: 50 / 450 [11%] (Warmup)
Chain 4: Iteration: 100 / 450 [22%] (Warmup)
Chain 3: Iteration: 70 / 450 [15%] (Warmup)
Chain 2: Iteration: 60 / 450 [13%] (Warmup)
Chain 2: Iteration: 70 / 450 [15%] (Warmup)
Chain 3: Iteration: 80 / 450 [17%] (Warmup)
Chain 4: Iteration: 110 / 450 [24%] (Warmup)
Chain 2: Iteration: 80 / 450 [17%] (Warmup)
Chain 3: Iteration: 90 / 450 [20%] (Warmup)
Chain 1: Iteration: 110 / 450 [24%] (Warmup)
Chain 3: Iteration: 100 / 450 [22%] (Warmup)
Chain 2: Iteration: 90 / 450 [20%] (Warmup)
Chain 3: Iteration: 110 / 450 [24%] (Warmup)
Chain 1: Iteration: 120 / 450 [26%] (Warmup)
Chain 4: Iteration: 120 / 450 [26%] (Warmup)
Chain 2: Iteration: 100 / 450 [22%] (Warmup)
Chain 1: Iteration: 130 / 450 [28%] (Warmup)
Chain 3: Iteration: 120 / 450 [26%] (Warmup)
Chain 4: Iteration: 130 / 450 [28%] (Warmup)
Chain 1: Iteration: 140 / 450 [31%] (Warmup)

Chain 3: Iteration: 130 / 450 [28%] (Warmup)
Chain 4: Iteration: 140 / 450 [31%] (Warmup)
Chain 3: Iteration: 140 / 450 [31%] (Warmup)
Chain 1: Iteration: 150 / 450 [33%] (Warmup)
Chain 1: Iteration: 151 / 450 [33%] (Sampling)
Chain 4: Iteration: 150 / 450 [33%] (Warmup)
Chain 4: Iteration: 151 / 450 [33%] (Sampling)
Chain 2: Iteration: 110 / 450 [24%] (Warmup)
Chain 3: Iteration: 150 / 450 [33%] (Warmup)
Chain 4: Iteration: 160 / 450 [35%] (Sampling)
Chain 3: Iteration: 151 / 450 [33%] (Sampling)
Chain 1: Iteration: 160 / 450 [35%] (Sampling)
Chain 4: Iteration: 170 / 450 [37%] (Sampling)
Chain 3: Iteration: 160 / 450 [35%] (Sampling)
Chain 1: Iteration: 170 / 450 [37%] (Sampling)
Chain 4: Iteration: 180 / 450 [40%] (Sampling)
Chain 3: Iteration: 170 / 450 [37%] (Sampling)
Chain 2: Iteration: 120 / 450 [26%] (Warmup)
Chain 4: Iteration: 190 / 450 [42%] (Sampling)
Chain 1: Iteration: 180 / 450 [40%] (Sampling)
Chain 2: Iteration: 130 / 450 [28%] (Warmup)
Chain 3: Iteration: 180 / 450 [40%] (Sampling)
Chain 4: Iteration: 200 / 450 [44%] (Sampling)
Chain 1: Iteration: 190 / 450 [42%] (Sampling)
Chain 2: Iteration: 140 / 450 [31%] (Warmup)
Chain 4: Iteration: 210 / 450 [46%] (Sampling)
Chain 2: Iteration: 150 / 450 [33%] (Warmup)
Chain 2: Iteration: 151 / 450 [33%] (Sampling)
Chain 3: Iteration: 190 / 450 [42%] (Sampling)
Chain 1: Iteration: 200 / 450 [44%] (Sampling)
Chain 2: Iteration: 160 / 450 [35%] (Sampling)
Chain 4: Iteration: 220 / 450 [48%] (Sampling)
Chain 3: Iteration: 200 / 450 [44%] (Sampling)
Chain 2: Iteration: 170 / 450 [37%] (Sampling)
Chain 1: Iteration: 210 / 450 [46%] (Sampling)
Chain 4: Iteration: 230 / 450 [51%] (Sampling)
Chain 2: Iteration: 180 / 450 [40%] (Sampling)
Chain 3: Iteration: 210 / 450 [46%] (Sampling)
Chain 2: Iteration: 190 / 450 [42%] (Sampling)
Chain 4: Iteration: 240 / 450 [53%] (Sampling)
Chain 1: Iteration: 220 / 450 [48%] (Sampling)
Chain 2: Iteration: 200 / 450 [44%] (Sampling)
Chain 3: Iteration: 220 / 450 [48%] (Sampling)

Chain 4: Iteration: 250 / 450 [55%] (Sampling)
Chain 2: Iteration: 210 / 450 [46%] (Sampling)
Chain 1: Iteration: 230 / 450 [51%] (Sampling)
Chain 3: Iteration: 230 / 450 [51%] (Sampling)
Chain 2: Iteration: 220 / 450 [48%] (Sampling)
Chain 4: Iteration: 260 / 450 [57%] (Sampling)
Chain 2: Iteration: 230 / 450 [51%] (Sampling)
Chain 1: Iteration: 240 / 450 [53%] (Sampling)
Chain 3: Iteration: 240 / 450 [53%] (Sampling)
Chain 4: Iteration: 270 / 450 [60%] (Sampling)
Chain 2: Iteration: 240 / 450 [53%] (Sampling)
Chain 2: Iteration: 250 / 450 [55%] (Sampling)
Chain 1: Iteration: 250 / 450 [55%] (Sampling)
Chain 3: Iteration: 250 / 450 [55%] (Sampling)
Chain 4: Iteration: 280 / 450 [62%] (Sampling)
Chain 2: Iteration: 260 / 450 [57%] (Sampling)
Chain 4: Iteration: 290 / 450 [64%] (Sampling)
Chain 2: Iteration: 270 / 450 [60%] (Sampling)
Chain 3: Iteration: 260 / 450 [57%] (Sampling)
Chain 1: Iteration: 260 / 450 [57%] (Sampling)
Chain 2: Iteration: 280 / 450 [62%] (Sampling)
Chain 4: Iteration: 300 / 450 [66%] (Sampling)
Chain 3: Iteration: 270 / 450 [60%] (Sampling)
Chain 2: Iteration: 290 / 450 [64%] (Sampling)
Chain 1: Iteration: 270 / 450 [60%] (Sampling)
Chain 2: Iteration: 300 / 450 [66%] (Sampling)
Chain 4: Iteration: 310 / 450 [68%] (Sampling)
Chain 3: Iteration: 280 / 450 [62%] (Sampling)
Chain 2: Iteration: 310 / 450 [68%] (Sampling)
Chain 1: Iteration: 280 / 450 [62%] (Sampling)
Chain 4: Iteration: 320 / 450 [71%] (Sampling)
Chain 2: Iteration: 320 / 450 [71%] (Sampling)
Chain 3: Iteration: 290 / 450 [64%] (Sampling)
Chain 1: Iteration: 290 / 450 [64%] (Sampling)
Chain 2: Iteration: 330 / 450 [73%] (Sampling)
Chain 4: Iteration: 330 / 450 [73%] (Sampling)
Chain 3: Iteration: 300 / 450 [66%] (Sampling)
Chain 2: Iteration: 340 / 450 [75%] (Sampling)
Chain 1: Iteration: 300 / 450 [66%] (Sampling)
Chain 4: Iteration: 340 / 450 [75%] (Sampling)
Chain 2: Iteration: 350 / 450 [77%] (Sampling)
Chain 3: Iteration: 310 / 450 [68%] (Sampling)
Chain 2: Iteration: 360 / 450 [80%] (Sampling)

```

Chain 1: Iteration: 310 / 450 [ 68%] (Sampling)
Chain 4: Iteration: 350 / 450 [ 77%] (Sampling)
Chain 3: Iteration: 320 / 450 [ 71%] (Sampling)
Chain 2: Iteration: 370 / 450 [ 82%] (Sampling)
Chain 4: Iteration: 360 / 450 [ 80%] (Sampling)
Chain 2: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 1: Iteration: 320 / 450 [ 71%] (Sampling)
Chain 3: Iteration: 330 / 450 [ 73%] (Sampling)
Chain 2: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 4: Iteration: 370 / 450 [ 82%] (Sampling)
Chain 1: Iteration: 330 / 450 [ 73%] (Sampling)
Chain 3: Iteration: 340 / 450 [ 75%] (Sampling)
Chain 2: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 4: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 2: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 3: Iteration: 350 / 450 [ 77%] (Sampling)
Chain 4: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 1: Iteration: 340 / 450 [ 75%] (Sampling)
Chain 2: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 2: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 4: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 3: Iteration: 360 / 450 [ 80%] (Sampling)
Chain 1: Iteration: 350 / 450 [ 77%] (Sampling)
Chain 2: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 4: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 3: Iteration: 370 / 450 [ 82%] (Sampling)
Chain 2: Iteration: 450 / 450 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 0.64 seconds (Warm-up)
Chain 2:           0.403 seconds (Sampling)
Chain 2:           1.043 seconds (Total)
Chain 2:
Chain 1: Iteration: 360 / 450 [ 80%] (Sampling)
Chain 3: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 4: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 1: Iteration: 370 / 450 [ 82%] (Sampling)
Chain 3: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 4: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 1: Iteration: 380 / 450 [ 84%] (Sampling)
Chain 4: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 3: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 1: Iteration: 390 / 450 [ 86%] (Sampling)
Chain 4: Iteration: 450 / 450 [100%] (Sampling)

```



```

Chain 4:
Chain 4: Elapsed Time: 0.501 seconds (Warm-up)
Chain 4:           0.595 seconds (Sampling)
Chain 4:           1.096 seconds (Total)
Chain 4:
Chain 3: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 1: Iteration: 400 / 450 [ 88%] (Sampling)
Chain 3: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 1: Iteration: 410 / 450 [ 91%] (Sampling)
Chain 3: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 1: Iteration: 420 / 450 [ 93%] (Sampling)
Chain 3: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 1: Iteration: 430 / 450 [ 95%] (Sampling)
Chain 3: Iteration: 450 / 450 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 0.52 seconds (Warm-up)
Chain 3:           0.678 seconds (Sampling)
Chain 3:           1.198 seconds (Total)
Chain 3:
Chain 1: Iteration: 440 / 450 [ 97%] (Sampling)
Chain 1: Iteration: 450 / 450 [100%] (Sampling)
Chain 1:
Chain 1: Elapsed Time: 0.552 seconds (Warm-up)
Chain 1:           0.737 seconds (Sampling)
Chain 1:           1.289 seconds (Total)
Chain 1:

```

Warning: There were 10 divergent transitions after warmup. See <https://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup> to find out why this is a problem and how to eliminate them.

Warning: Examine the pairs() plot to diagnose sampling problems

Warning: Tail Effective Samples Size (ESS) is too low, indicating posterior variances and tail quantiles may not be accurate. Running the chains for more iterations may help. See <https://mc-stan.org/misc/warnings.html#tail-ess>

Extracting posteriors...

```
countryplot <- bayescoveragemodel::plot_estimates_local_all(
  results = fit,
  results2 = fit_noroutine,
  modelnames = c("with routine", "without routine"),
  save_plots = FALSE)[[1]]
countryplot
```

